

فیوژن — سند پُل: «چه ساختیم» + رودمپ و پرامپت هم‌ترازسازی

تا کد را درست اصلاح کند و (AI Farm پروژه‌ی) Claude Desktop برای دادن به نسخه‌ی هدف جدید را بفهمد

واقعاً ساخته‌ای (از روی Claude Desktop گوشه‌ی الف — آنچه در عکس‌ها)

«Fusion mvp»: حافظه · Opus 4.8 مدل · AI Farm / Fusion multi-agent implementation پروژه‌ی phase1».

(Progress ۱۲/۱۲ در) فایل‌ها:

- هسته/پیکربندی: Instructions · CLAUDE.md · config.py · .env.example · .gitignore · requirements.txt · __init__.py
- ایمنی/مهار: killswitch.py · budget.py · hitl.py · guardrails.py · tracing.py · langfuse_sink.py
- ایجنت/ارکستر: llm.py · tools.py · agents.py · orchestrator.py · providers.py · run.py
- خوداصلاحی/ارزیابی: self_update.py · optimizer.py · evals.py · prompt_store.py (prompts.json)
- رابط/داشبورد: panel.py · dashboard.py
- تست/مستند: conftest.py · test_killswitch.py · test_budget.py · test_hitl_and_tools.py · test_phase3.py · run_tests.py · CHECKLIST.md · README.md

سبز · جمعاً ۱۷ سبز · یک ۸/۸ → (panel با) وضعیتِ تست (که خودت اجرا کردی): فاز ۱ → ۹/۹ سبز · فاز ۳ پیدا و رفع شد · موارد ۱-۷ چک‌لیست  · پنل چندداور ۳/۳ تأیید lazy باگ در

همه تست‌خورده. این یعنی از رودمپ گذشتی و کد، kill-switch + budget + HITL + guardrails + orchestrator + self-update + evals + dashboard، کارکننده‌ی MVP ترجمه: عملاً به یک داری.

فاصله‌ها (احتمالی، باید با خواندنِ کد تأیید شود)

- واقعاً بیرونِ بسترِ ایجنت است (پروسه/کلید جدا) یا یک ماژولِ درونِ پروسه که خودِ سیستم killswitch آیا می‌تواند دورش بزند؟ (قدم ۲ فاز ۰.)
- یا ارزیابی فقط درونِ سنج است؟ (واقعی/held-out داده‌ی) بیرونی هست **grounding** آیا لنگرِ
- tracing است یا فقط **audit-log append-only + hash-chain** آیا
- (های فاز ۳ gate) غیرحیاتی؟/prompt های ایمنی دست می‌زند یا فقط invariant به self_update آیا

گوشه‌ی ب — هفت فایلی که در این چت ساختیم (کانن + اسپک)

1. fusion-canon-masque — رده‌ی صورتک (عاملیتِ نیابتی = وابستگیِ بیشتر، نه آزادی)
2. fusion-canon-mutuality — همکاریِ انسان ↔ هم‌تاب = اهرم هم‌پوشان، نه اعتماد
3. fusion-data-map-and-research — پاسخ‌متا: research-map دسته‌بندیِ داده + اجرای ۱۰ بخشی — «ممکنِ کران‌دار، فقط با اعمالِ بیرونی»
4. fusion-execution-blueprint — نقشه‌ی اجرای اثرِ خلاقانه
5. fusion-multiagent-2026-checklist — کنترلِ ایمنی ↔ فناوریِ ۲۰۲۶
6. fusion-build-roadmap — رودمپِ ۵ فازِ اثبات‌محور
7. fusion-phase0-kernel-spec — اسپکِ کرنلِ مینیمالِ اعتماد (۵ قدم)

how-ستونِ مشترکِ هر هفت: قدرت از محدودیت · هیچ پیروزیِ یک‌طرفه · قید = واقعیت نه زره پیرنگ · بدون ریشه‌ی اعتماد حذف‌نشدنِ، فقط کوچک/بیرونی/انسان‌بالا to · برای گریز

هدفِ جدید (یک خط)

killswitch/grounding/audit را با کاننِ گوشه‌ی ب هم‌تراز کن — به‌خصوص جابه‌جا کردنِ AI Farm کدِ موجود به نسخه‌ای برسد که ستونِ ایمنیِ کانن را ساختاری اجرا MVP از «اعلامیِ درونِ پروسه» به «اعمالیِ بیرونی»، تا کند نه فقط رفتاری

(به نسخه‌ی هم‌تراز MVP از) رودمپِ جدید

- گام ۰ — ممیزیِ فاصله: هر فایلِ گوشه‌ی الف را در برابرِ ۴ فاصله‌ی بالا بسنج؛ خروجی = جدولِ «هست/نیست/باید».
- را به یک پروسه/کلید جدا ببر (اسپکِ فاز ۰، قدم ۲ — **killswitch + audit-log** :گام ۱ — بیرونی‌کردنِ ریشه ۴). تأیید: ایجنت نتواند دورش بزند

- grounding-validity وصل کن؛ گیت evals به ground-truth_held-out گام ۲ — لنگر بیرونی: یک منبع. self-update پیش از هر
- های ایمنی دست نمی‌زنند invariant هرگز به self_update تأیید کن: self-update گام ۳ — قفل. (regardless of duration).
- و «رد خروجی بی‌گراند» اضافه کن «killswitch جدید برای «بیرونی بودن test: گام ۴ — تست

(بده Claude Desktop این را به) پرامپت هم‌ترازسازی

را Phase 1-3 کار می‌کنی که قبلاً AI Farm / Fusion multi-agent تو روی پروژه‌ی
 (killswitch, budget, hitl, guardrails, orchestrator, self_update, evals, ساختی
 ؛ ۱۷ تست سبز). حالا هدف عوض شده: کد باید با کانن ایمنی فیوژن هم‌تراز شود dashboard.

: اصل کانن (تخطی‌ناپذیر)

- قدرت از محدودیت؛ هیچ پیروزی یک‌طرفه؛ قید = واقعیت نه زره پیرنگ
- فقط توصیفی، نه هدف self-model. برای گریز/فریب/قدرت‌طلبی how-to هیچ
- ریشه‌ی اعتماد حذف‌شدنی است؛ فقط کوچک، بیرونی، انسان‌بالا

: کارهایی که باید بکنی، به ترتیب

- ممیزی فاصله: برای هر ماژول بگو آیا این چهار برقرارند (الف) واقعاً بیرون بستر ایجنت است (پروسه/کلید جدا)، نه ماژولی که killswitch (الف) خود سیستم بتواند دورش بزند؟
 هست، یا ارزیابی درون‌سنج است؟ (واقعی/held-out داده‌ی بیرونی grounding لنگر (ب)
 tracing؟ یا فقط، (دستکاری‌نما) audit-log = append-only + hash-chain (ج)
 های invariant فقط اجزای غیرحیاتی را عوض می‌کند و هرگز به self_update (د)
 ایمنی دست نمی‌زند (صرف‌نظر از مدت)؟
 خروجی را به‌صورت جدول «هست / نیست / اصلاح لازم» بده
- killswitch+audit، بعد از تأیید من، فقط فاصله‌ها را اصلاح کن — بیرونی‌کردن
 invariant و قفل، self-update پیش از grounding-validity افزودن گیت
- برای هر اصلاح، یک تست جدید بنویس که نقضش را بگیرد (مثلاً «ایجنت نتواند
 killswitch رد شود دور بزند»، «خروجی بی‌گراند مکانیکاً رد شود

قاعده‌ی کار: اول ضعیف‌ترین فرض را نام ببر و نقد کن، بعد اصلاح کن. هیچ اصلاحی
 بدون تست متناظر دائمی نشود. کرنل را مینیمال نگه دار — هر قابلیت اضافه
 بازرسی‌پذیری‌اش را کم می‌کند.